Universidad / Proyecto Académico

Especificación de Requisitos del Software (ERS)

Mi Playa para Hoy"

Aplicación de recomendación de playas basada en preferencias, condiciones meteorológicas y contexto del usuario

---

Asignatura / Proyecto:	Producción de Software / Proyecto de desarrollo
Metodología:	SCRUM
Equipo:	6 integrantes
Lenguaje principal:	Python
Versión del documento:	1.0
Fecha:	11 de marzo de 2026

Documento elaborado para definir el alcance funcional y técnico del sistema, así como su planificación inicial en tres sprints.

Control de versiones

Versión	Fecha	Autoría	Descripción
1.0	11 de marzo de 2026	Equipo del proyecto	Primera versión del documento ERS para la aplicación *¡Mi playita para HOY!*.

Índice

Control de versiones **1**

1. Introducción **3**
1.1. Propósito del documento . . . 3
1.2. Alcance del producto . . . 3
1.3. Definiciones, acrónimos y abreviaturas . . . 3
1.4. Visión general del documento . . . 4

2. Descripción general del sistema **4**
2.1. Perspectiva del producto . . . 4
2.2. Objetivos del sistema . . . 4
2.3. Tipos de usuarios . . . 5
2.4. Suposiciones y dependencias . . . 5

3. Decisiones iniciales de arquitectura y tecnología **5**
3.1. Lenguaje principal . . . 5
3.2. Base de datos recomendada . . . 6
3.3. Servicios externos previstos . . . 6

4. Requisitos específicos **6**
4.1. Requisitos funcionales . . . 6
4.1.1. Gestión de consulta de recomendaciones . . . 6
4.1.2. Información detallada de playas . . . 7
4.1.3. Gestión de usuario . . . 7
4.1.4. Reseñas y aportaciones de usuarios . . . 8
4.1.5. Chatbot y funciones inteligentes . . . 8
4.1.6. Administración . . . 8
4.2. Requisitos no funcionales . . . 9

5. Reglas de negocio **9**

6. Modelo conceptual de datos **10**
6.1. Entidades principales 10
6.2. Atributos iniciales de playa 10
6.3. Variables dinámicas previstas 11

7. Casos de uso principales **12**
7.1. CU-01: Obtener recomendaciones de playas 12
7.2. CU-02: Guardar playa en favoritas 12
7.3. CU-03: Publicar reseña temporal 12

8. Historias de usuario **13**
8.1. Historias de usuario funcionales 13
8.2. Criterios de aceptación de ejemplo 14

9. Historias técnicas **14**

10. Priorización del producto **15**
10.1. MVP 15

11. Planificación SCRUM en tres sprints **16**
11.1. Sprint 1: Base del producto y consulta inicial 16
11.2. Sprint 2: Filtros avanzados, cuentas y favoritas 16
11.3. Sprint 3: Reseñas, calidad final y funciones evolutivas 17

12. Reparto orientativo del trabajo para un equipo de seis personas **17**

13. Riesgos iniciales del proyecto **18**

14. Conclusiones **18**

A. Resumen de backlog inicial **18**

1. Introducción

1.1. Propósito del documento

El presente documento tiene como objetivo definir de manera formal los requisitos del sistema *¡Mi playita para HOY!*, una aplicación orientada a recomendar al usuario las tres mejores playas para la actividad que desea realizar, atendiendo a sus preferencias, su ubicación y a diversas variables ambientales y de contexto.

Además de recoger requisitos funcionales y no funcionales, este documento incluye una propuesta inicial de historias de usuario, historias técnicas y una planificación de alto nivel del desarrollo siguiendo la metodología SCRUM en tres sprints.

1.2. Alcance del producto

¡Mi playita para HOY! será una aplicación software que permitirá al usuario:

- seleccionar una actividad a realizar en la playa;
- indicar filtros o preferencias relevantes;
- consultar playas recomendadas para un día y hora concretos;
- visualizar un *top-3* de playas ordenadas según adecuación;
- acceder a información detallada de cada playa;
- guardar favoritas y, opcionalmente, publicar reseñas temporales.

El sistema integrará información meteorológica y ambiental procedente de servicios externos, siendo OpenMeteo la fuente principal prevista para variables climáticas.

1.3. Definiciones, acrónimos y abreviaturas

Término	**Definición**
ERS	Especificación de Requisitos del Software.
SCRUM	Marco de trabajo ágil basado en iteraciones cortas y entregables incrementales.
Sprint	Iteración temporal dentro de SCRUM.
Top-3	Las tres mejores recomendaciones calculadas por el sistema.
OpenMeteo	Servicio externo para obtención de información meteorológica.

Geolocalización	Obtención de la ubicación del usuario para calcular distancia a playas.
Review / reseña	Comentario o valoración aportada por usuarios sobre una playa.
Calima	Presencia de polvo en suspensión que afecta la calidad ambiental y la experiencia en playa.

1.4. Visión general del documento

El documento se estructura en una descripción general del sistema, la definición detallada de requisitos, las reglas de negocio, el modelo conceptual, el backlog inicial de historias de usuario e historias técnicas y una propuesta de organización del trabajo en tres sprints.

2. Descripción general del sistema

2.1. Perspectiva del producto

La aplicación se concibe como un recomendador inteligente de playas. No se limita a mostrar información estática, sino que combina:

- datos de playas y sus características permanentes;
- datos dinámicos meteorológicos y ambientales;
- preferencias explícitas del usuario;
- un mecanismo de ranking que ordena resultados según adecuación.

El sistema tendrá una arquitectura cliente-servidor, con un *frontend* web o móvil y un *backend* desarrollado en Python. El backend será responsable de consultar APIs externas, almacenar datos internos, aplicar reglas de negocio y calcular recomendaciones.

2.2. Objetivos del sistema

Los objetivos principales del producto son:

- facilitar la elección de una playa adecuada para una actividad concreta;
- reducir el tiempo de búsqueda y comparación manual;
- adaptar las recomendaciones al contexto real del usuario;
- ofrecer una experiencia sencilla, útil y visual;

- sentar una base técnica escalable para futuras mejoras como chatbot o análisis de imágenes.

2.3. Tipos de usuarios

Se identifican los siguientes perfiles:

- **Usuario anónimo**: consulta recomendaciones básicas sin guardar preferencias permanentes.
- **Usuario registrado**: puede guardar favoritas, publicar reseñas y mantener preferencias.
- **Administrador**: gestiona playas, datos estáticos, moderación de reseñas y configuración general.

2.4. Suposiciones y dependencias

- La aplicación dependerá de servicios externos para obtener datos meteorológicos actualizados.
- La calidad de las recomendaciones estará condicionada por la disponibilidad y precisión de esos datos.
- Algunas características de playa (por ejemplo, escuelas de surf, puestos de comida o tipo de arena) se almacenarán en base de datos propia.
- La geolocalización del usuario requerirá su consentimiento.
- Las reseñas temporales se plantean como funcionalidad secundaria y podrán limitarse en alcance en los primeros sprints.

3. Decisiones iniciales de arquitectura y tecnología

3.1. Lenguaje principal

El lenguaje principal del backend será **Python**, por su rapidez de desarrollo, su ecosistema maduro y su adecuación para lógica de negocio, integración de APIs y futura incorporación de componentes inteligentes.

3.2. Base de datos recomendada

Se recomienda utilizar una **base de datos SQL**, preferiblemente **PostgreSQL**, por las siguientes razones:

- estructura clara para entidades como usuarios, playas, favoritas, reseñas y actividades;
- facilidad para relaciones entre tablas;
- consistencia transaccional;
- mejor adecuación para consultas complejas y filtrado combinado;
- posibilidad de incorporar capacidades geoespaciales mediante PostGIS en evoluciones posteriores.

Firebase podría valorarse para autenticación o almacenamiento auxiliar, pero no se recomienda como núcleo principal del dominio si el sistema debe calcular rankings complejos y soportar reglas relacionales con claridad.

3.3. Servicios externos previstos

- Servicio meteorológico principal: OpenMeteo.
- Servicio de geolocalización del navegador o dispositivo.
- Servicio opcional futuro para análisis de imágenes con IA.

4. Requisitos específicos

4.1. Requisitos funcionales

4.1.1. Gestión de consulta de recomendaciones

ID	Descripción
RF-01	El sistema debe permitir al usuario seleccionar una actividad principal que desea realizar en la playa.
RF-02	El sistema debe permitir al usuario seleccionar el día para el cual desea obtener recomendaciones.
RF-03	El sistema debe permitir al usuario seleccionar una hora concreta o una franja horaria.

RF-04	El sistema debe permitir filtrar resultados por distancia respecto a la ubicación del usuario.
RF-05	El sistema debe permitir aplicar filtros sobre variables como viento, oleaje, temperatura, radiación UV, precipitaciones, nubosidad y marea.
RF-06	El sistema debe permitir indicar preferencias estructurales de la playa, como arena o piedra.
RF-07	El sistema debe permitir indicar preferencias adicionales, como presencia de escuelas de surf/windsurf, puestos de comida cercanos o existencia de charcos naturales.
RF-08	El sistema debe calcular y mostrar un top-3 de playas ordenadas por adecuación a la actividad y a las preferencias del usuario.
RF-09	El sistema debe mostrar para cada recomendación una puntuación o nivel de adecuación interpretable.
RF-10	El sistema debe permitir acceder a la ficha detallada de una playa recomendada.

4.1.2. Información detallada de playas

ID	Descripción
RF-11	El sistema debe mostrar información general de la playa: nombre, ubicación, tipo, accesibilidad y servicios disponibles.
RF-12	El sistema debe mostrar las condiciones estimadas para la fecha y hora seleccionadas.
RF-13	El sistema debe mostrar iconos de actividades recomendadas o compatibles con la playa.
RF-14	El sistema debe indicar si la playa es apta para familia, mascotas, surf, windsurf, paddle surf, kayak u otras actividades definidas.
RF-15	El sistema debe permitir visualizar si la playa dispone de escuelas de surf o windsurf cercanas.

4.1.3. Gestión de usuario

ID	Descripción
RF-16	El sistema debe permitir el registro de usuarios.
RF-17	El sistema debe permitir el inicio y cierre de sesión.

RF-18	El sistema debe permitir a usuarios registrados guardar playas como favoritas.
RF-19	El sistema debe permitir a usuarios registrados consultar su listado de playas favoritas.
RF-20	El sistema debe permitir a usuarios registrados gestionar preferencias persistentes básicas.

4.1.4. Reseñas y aportaciones de usuarios

ID	Descripción
RF-21	El sistema debe permitir a usuarios registrados publicar reseñas sobre una playa.
RF-22	El sistema podrá permitir adjuntar imágenes a una reseña, sujeto a restricciones de alcance y moderación.
RF-23	El sistema debe permitir definir una caducidad temporal para determinadas reseñas o reportes recientes.
RF-24	El sistema debe impedir mostrar reseñas expiradas como información actual.
RF-25	El sistema debe permitir al administrador moderar o eliminar contenido inapropiado.

4.1.5. Chatbot y funciones inteligentes

ID	Descripción
RF-26	El sistema podrá incluir un chatbot que permita al usuario expresar su necesidad en lenguaje natural.
RF-27	El chatbot deberá traducir la consulta del usuario en filtros o criterios interpretables por el motor de recomendación.
RF-28	El sistema podrá incorporar en el futuro un módulo de IA para analizar imágenes aportadas por usuarios.

4.1.6. Administración

ID	Descripción

RF-29	El sistema debe permitir al administrador crear, modificar y desactivar playas registradas.
RF-30	El sistema debe permitir al administrador actualizar atributos estáticos de las playas.
RF-31	El sistema debe permitir al administrador revisar la calidad o disponibilidad de los datos externos integrados.

4.2. Requisitos no funcionales

ID	Descripción
RNF-01	El sistema debe presentar una interfaz clara, visual e intuitiva, orientada a usuarios no técnicos.
RNF-02	El tiempo de respuesta de una consulta de recomendación no debería superar los 3 segundos en condiciones normales de uso.
RNF-03	El sistema debe ser modular para facilitar la incorporación futura de nuevas actividades, playas o fuentes de datos.
RNF-04	El backend debe desarrollarse en Python con separación clara entre capas de presentación, negocio y persistencia.
RNF-05	El sistema debe garantizar almacenamiento seguro de credenciales mediante mecanismos adecuados de hash y autenticación.
RNF-06	El sistema debe cumplir la normativa de protección de datos aplicable respecto al tratamiento de datos personales y geolocalización.
RNF-07	El sistema debe manejar fallos de APIs externas de forma controlada, mostrando mensajes apropiados al usuario.
RNF-08	El sistema debe poder desplegarse en un entorno reproducible y documentado.
RNF-09	El sistema debe permitir pruebas unitarias e integradas sobre la lógica principal de recomendación.
RNF-10	La aplicación debe ser responsive o adaptable al dispositivo objetivo.

5. Reglas de negocio

RN-1 El sistema siempre debe devolver como máximo tres playas en el ranking principal.

RN-2 Si no existen tres playas que cumplan estrictamente todos los filtros, el sistema podrá relajar filtros secundarios e informar de ello al usuario.

RN-3 La distancia debe calcularse a partir de la geolocalización del usuario siempre que este haya autorizado dicho acceso.

RN-4 Las variables meteorológicas y ambientales se considerarán dinámicas y deberán asociarse a un instante temporal concreto.

RN-5 La adecuación de una playa a una actividad dependerá de una combinación ponderada de parámetros, no de un único valor.

RN-6 Las reseñas temporales se considerarán válidas solo hasta la fecha/hora de expiración definida.

RN-7 Solo los usuarios autenticados podrán guardar favoritas o publicar reseñas.

RN-8 Las funcionalidades de chatbot e IA podrán quedar etiquetadas como alcance opcional o evolutivo si el tiempo de proyecto lo exige.

6. Modelo conceptual de datos

6.1. Entidades principales

Se identifican, como mínimo, las siguientes entidades:

- **Usuario**: información de cuenta, credenciales, preferencias.
- **Playa**: nombre, localización, tipo, servicios, accesibilidad, características.
- **Actividad**: catálogo de actividades soportadas por el sistema.
- **Condición ambiental**: valores meteorológicos y ambientales por playa, fecha y hora.
- **Favorita**: relación entre usuario y playa.
- **Reseña**: comentario publicado por usuario sobre playa.
- **Servicio cercano**: escuelas, puestos de comida u otros recursos próximos.

6.2. Atributos iniciales de playa

Cada playa podrá contener, entre otros, los siguientes atributos:

- nombre;
- coordenadas;
- municipio o zona;

- tipo de superficie (arena, piedra, mixta);
- apta para mascotas;
- apta para familias;
- apta para surf;
- apta para windsurf;
- apta para paddle surf;
- apta para kayak;
- posibilidad de acampar;
- disponibilidad de espacios deportivos;
- existencia de charcos naturales;
- puestos de comida cercanos;
- escuelas de surf/windsurf cercanas.

6.3. Variables dinámicas previstas

Las condiciones dinámicas consideradas por el motor de recomendación incluyen:

- temperatura ambiente;
- temperatura del agua;
- viento;
- oleaje;
- precipitaciones o probabilidad de lluvia;
- índice ultravioleta;
- partículas de polvo en suspensión o calima;
- estado de marea;
- nubosidad.

7. Casos de uso principales

7.1. CU-01: Obtener recomendaciones de playas

Actor principal: usuario

Descripción: el usuario selecciona una actividad, un día, una hora y una serie de preferencias o filtros. El sistema procesa la consulta y devuelve las tres playas más adecuadas.

Flujo principal:

1. El usuario accede al buscador de playas.
2. El usuario selecciona una actividad.
3. El usuario selecciona día y hora.
4. El usuario define filtros opcionales.
5. El sistema obtiene datos necesarios.
6. El sistema calcula la puntuación de adecuación.
7. El sistema muestra el top-3 de playas recomendadas.

Flujos alternativos:

- Si no hay geolocalización disponible, el sistema solicita ubicación manual o desactiva el filtro de distancia.
- Si la API externa falla, el sistema informa del problema y usa el último dato válido disponible si existe.

7.2. CU-02: Guardar playa en favoritas

Actor principal: usuario registrado

Descripción: el usuario autenticado guarda una playa para consultarla posteriormente.

7.3. CU-03: Publicar reseña temporal

Actor principal: usuario registrado

Descripción: el usuario añade una reseña sobre el estado reciente de una playa. La reseña caduca después de un periodo establecido.

8. Historias de usuario

8.1. Historias de usuario funcionales

ID	Historia	Prioridad	Puntos
HU-01	Como usuario, quiero seleccionar una actividad para recibir recomendaciones de playas adecuadas.	Alta	3
HU-02	Como usuario, quiero seleccionar el día de mi salida a la playa para obtener resultados coherentes con esa fecha.	Alta	3
HU-03	Como usuario, quiero seleccionar una hora concreta para afinar la recomendación.	Alta	3
HU-04	Como usuario, quiero filtrar por distancia desde mi ubicación para no desplazarme demasiado **o desde una ubicacion seleccionada.**.	Alta	5
HU-05	Como usuario, quiero filtrar por variables como viento, oleaje o temperatura para adaptar la recomendación a mis necesidades.	Alta	5
HU-06	Como usuario, quiero ver el top-3 de playas recomendadas para no tener que revisar una lista enorme**, aunque podria ver más.**.	Alta	5
HU-07	Como usuario, quiero consultar una ficha detallada de cada playa recomendada para entender por qué ha sido propuesta.	Alta	3
HU-08	Como usuario, quiero identificar visualmente qué actividades son apropiadas en cada playa mediante iconos.	Media	2
HU-09	Como usuario, quiero filtrar por tipo de playa (arena o piedra) para elegir el entorno que prefiero.	Media	2
HU-10	Como usuario, quiero saber si una playa tiene escuelas de surf o windsurf cerca.	Media	3
HU-11	Como usuario, quiero saber si una playa tiene puestos de comida cerca.	Media	2
HU-12	Como usuario, quiero saber si una playa tiene charcos naturales.	Baja	2
HU-13	Como usuario, quiero registrarme para guardar mis playas favoritas.	Alta	5
HU-14	Como usuario registrado, quiero iniciar sesión para acceder a mis preferencias y favoritas.	Alta	3
HU-15	Como usuario registrado, quiero guardar una playa en favoritas para consultarla más tarde.	Alta	3
HU-16	Como usuario registrado, quiero ver mi lista de playas favoritas.	Media	2

HU-17	Como usuario registrado, quiero publicar una reseña reciente sobre una playa.	Media	5
HU-18	Como usuario, quiero que las reseñas recientes desaparezcan tras un tiempo para que la información actual no quede desfasada.	Media	5
HU-19	Como usuario, quiero recibir resultados aunque algunos filtros no se cumplan al 100 %, siempre que el sistema me lo indique claramente.	Media	3
HU-20	Como usuario, quiero poder expresar en lenguaje natural lo que busco para no tener que ajustar cada filtro manualmente.	Baja	8

8.2. Criterios de aceptación de ejemplo

HU-06: Ver top-3 de playas recomendadas

- Dado un usuario con actividad, fecha y hora seleccionadas,
- cuando solicite recomendaciones,
- entonces el sistema deberá mostrar un máximo de tres playas ordenadas por puntuación.

HU-15: Guardar playa en favoritas

- Dado un usuario autenticado en la ficha de una playa,
- cuando pulse la opción de guardar en favoritas,
- entonces el sistema deberá registrar la relación y reflejar el cambio visualmente.

9. Historias técnicas

ID	Historia técnica	Prioridad	Puntos
HT-01	Como equipo de desarrollo, queremos definir la arquitectura base del proyecto para separar frontend, backend.	Alta	5
HT-02	Como equipo, queremos diseñar el esquema relacional de base de datos para soportar playas, usuarios, favoritas, actividades y reseñas.	Alta	5

HT-03	Como equipo, queremos implementar la integración con OpenMeteo para recuperar datos meteorológicos por ubicación y tiempo.	Alta	8
HT-04	Como equipo, queremos construir un motor de puntuación configurable para calcular la adecuación de cada playa a una actividad.	Alta	8
HT-05	Como equipo, queremos implementar autenticación segura de usuarios.	Alta	5
HT-06	Como equipo, queremos desarrollar una API interna para exponer búsquedas, recomendaciones, playas y favoritas.	Alta	8
HT-07	Como equipo, queremos poblar una base inicial de playas y atributos estáticos para disponer de datos de prueba reales o simulados.	Alta	5
HT-08	Como equipo, queremos implementar manejo de errores y tolerancia a fallos de servicios externos.	Alta	5
HT-09	Como equipo, queremos añadir pruebas unitarias para la lógica de recomendación.	Alta	5
HT-10	Como equipo, queremos añadir pruebas de integración sobre la comunicación con servicios externos y base de datos.	Media	5
HT-11	Como equipo, queremos desplegar el proyecto en un entorno reproducible para facilitar la validación por parte del profesorado o cliente.	Media	5
HT-12	Como equipo, queremos documentar endpoints, estructura del proyecto y procedimientos de instalación.	Alta	3
HT-13	Como equipo, queremos implementar expiración automática de reseñas temporales.	Media	5
HT-14	Como equipo, queremos estudiar la viabilidad de una interfaz de lenguaje natural para traducir consultas a filtros internos.	Baja	8
HT-15	Como equipo, queremos evaluar un módulo futuro de análisis de imágenes con IA para enriquecer la aplicación.	Baja	8

10. Priorización del producto

10.1. MVP

Se considera **Producto Mínimo Viable (MVP)** el conjunto de funcionalidades que permite:

- seleccionar actividad, día y hora;
- consultar datos de playas;
- aplicar filtros básicos;
- calcular y mostrar el top-3;
- visualizar información detallada de playa;
- registrar usuarios e incluir favoritas.

Las reseñas temporales y el chatbot se consideran alcance posterior o secundario si el tiempo del proyecto es limitado.

11. Planificación SCRUM en tres sprints

11.1. Sprint 1: Base del producto y consulta inicial

Objetivo: construir el esqueleto funcional del sistema y obtener recomendaciones básicas.

Historias candidatas:

- HU-01, HU-02, HU-03, HU-06, HU-07
- HT-01, HT-02, HT-03, HT-06, HT-07

Entregable esperado:

- estructura base del proyecto;
- base de datos inicial;
- integración inicial con OpenMeteo;
- buscador con actividad, día y hora;
- primer ranking top-3 con ficha de playa.

11.2. Sprint 2: Filtros avanzados, cuentas y favoritas

Objetivo: mejorar personalización y persistencia del usuario.

Historias candidatas:

- HU-04, HU-05, HU-08, HU-09, HU-10, HU-11, HU-13, HU-14, HU-15, HU-16
- HT-04, HT-05, HT-08, HT-09, HT-12

Entregable esperado:

- filtros avanzados;
- gestión de usuarios;
- favoritas;
- mejora del algoritmo de recomendación;
- pruebas básicas y documentación inicial.

11.3. Sprint 3: Reseñas, calidad final y funciones evolutivas

Objetivo: cerrar el producto base, añadir valor diferencial y reforzar robustez.

Historias candidatas:

- HU-17, HU-18, HU-19, HU-20
- HT-10, HT-11, HT-13, HT-14, HT-15

Entregable esperado:

- reseñas temporales;
- endurecimiento técnico;
- despliegue;
- estudio o prototipo del chatbot;
- valoración de la IA para imágenes como línea futura.

12. Reparto orientativo del trabajo para un equipo de seis personas

A nivel inicial, y sin perjuicio de ajustes posteriores, puede proponerse una distribución por áreas:

- **Persona 1**: coordinación SCRUM, backlog, documentación y apoyo funcional.
- **Persona 2**: backend, endpoints principales y lógica de negocio.
- **Persona 3**: integración con API meteorológica y tratamiento de datos externos.
- **Persona 4**: base de datos, persistencia y consultas.
- **Persona 5**: frontend e interfaz de usuario.

- **Persona 6**: testing, calidad, despliegue y soporte transversal.

Este reparto no debe entenderse como rígido; en SCRUM el trabajo debe adaptarse a la capacidad real del equipo en cada sprint.

13. Riesgos iniciales del proyecto

- Dependencia elevada de servicios externos para datos meteorológicos.
- Complejidad de definir una fórmula de recomendación realista y útil.
- Riesgo de sobredimensionar el alcance incluyendo chatbot, IA y reseñas multimedia demasiado pronto.
- Necesidad de mantener datos estáticos de playas con suficiente calidad.
- Gestión de protección de datos si se almacenan cuentas, preferencias y geolocalización.

14. Conclusiones

¡Mi playita para HOY! plantea una solución con valor práctico claro: ayudar al usuario a decidir a qué playa acudir en función de una actividad concreta y de condiciones reales. Desde el punto de vista de ingeniería del software, el proyecto resulta adecuado para un desarrollo iterativo en SCRUM, ya que permite definir un MVP sólido y añadir progresivamente filtros avanzados, funcionalidades sociales e incluso componentes inteligentes.

La decisión técnica más razonable para el núcleo del sistema es un backend en Python con una base de datos SQL, reservando integraciones más avanzadas como chatbot o análisis de imágenes para fases posteriores o como funcionalidades de mayor valor añadido una vez consolidado el flujo principal de recomendación.

A. Resumen de backlog inicial

ID	Elemento	Tipo	Prioridad	Puntos
HU-01	Selección de actividad	HU	Alta	3
HU-02	Selección de día	HU	Alta	3
HU-03	Selección de hora	HU	Alta	3
HU-04	Filtro por distancia	HU	Alta	5

HU-05	Filtros meteorológicos y ambientales	HU	Alta	5
HU-06	Top-3 de recomendaciones	HU	Alta	5
HU-07	Ficha detallada de playa	HU	Alta	3
HU-13	Registro de usuario	HU	Alta	5
HU-14	Inicio de sesión	HU	Alta	3
HU-15	Guardar favoritas	HU	Alta	3
HT-01	Arquitectura base	HT	Alta	5
HT-02	Modelo de base de datos	HT	Alta	5
HT-03	Integración OpenMeteo	HT	Alta	8
HT-04	Motor de puntuación	HT	Alta	8
HT-05	Autenticación segura	HT	Alta	5
HT-06	API interna	HT	Alta	8
HT-07	Dataset inicial de playas	HT	Alta	5
HT-09	Pruebas unitarias	HT	Alta	5